PRINCIPIOS SOLID

Principios de Diseño Orientado a Objetos

Prof. Esteban Santini · 2026

REPASO — LOS 4 PILARES DE LA POO

Antes de ver SOLID, repasemos brevemente los cuatro pilares que sostienen el paradigma orientado a objetos.

E

Encapsulamiento

Agrupar datos y comportamiento en una clase, ocultando los detalles internos.

A

Abstracción

Modelar solo lo esencial de un objeto, ignorando los detalles irrelevantes al contexto.

H

Herencia

Reutilizar y especializar comportamiento a partir de una clase más general.

P

Polimorfismo

Un mismo mensaje produce comportamientos distintos según el objeto que lo recibe.

¿QUÉ ES SOLID?

SOLID es un acrónimo que agrupa cinco principios de diseño orientado a objetos, formulados por Robert C. Martin ('Uncle Bob') y popularizados bajo ese acrónimo por Michael Feathers. Su objetivo es reducir el acoplamiento y facilitar que el software crezca sin romper lo que ya funciona.

S	O	L	I	D
Single Responsibility	**Open / Closed**	**Liskov Substitution**	**Interface Segregation**	**Dependency Inversion**
Principio de Responsabilidad Única	*Principio Abierto / Cerrado*	*Principio de Sustitución de Liskov*	*Principio de Segregación de Interfaces*	*Principio de Inversión de Dependencias*

SRP — DEFINICIÓN Y RELACIÓN CON LA POO

Principio de Responsabilidad Única (Single Responsibility Principle)

Una clase debe tener una única razón para cambiar: una sola responsabilidad claramente definida.

RELACIÓN CON LOS 4 PILARES DE LA POO

E **Encapsulamiento** *(RELACIÓN FUERTE)*
Agrupa en una clase solo los datos y métodos que cambian juntos; oculta el resto.

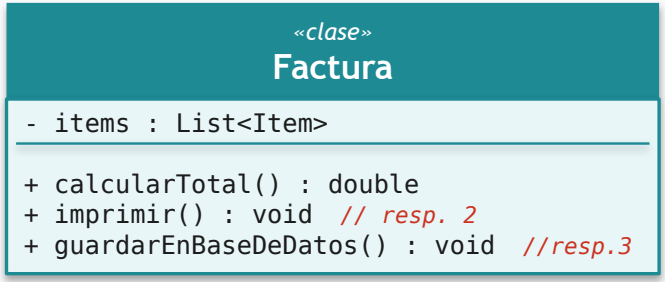
A **Abstracción** *(RELACIÓN MODERADA)*
Cada clase termina modelando un único concepto, claro y cohesivo.

H **Herencia** *(RELACIÓN LEVE)*
Evita bases "todo terreno" que arrastran responsabilidades ajenas a sus hijas.

P **Polimorfismo** *(RELACIÓN LEVE)*
Con responsabilidades acotadas, es más fácil sustituir una implementación por otra.

Ejemplo: una clase Factura que calcula, imprime y guarda datos tiene tres razones para cambiar.

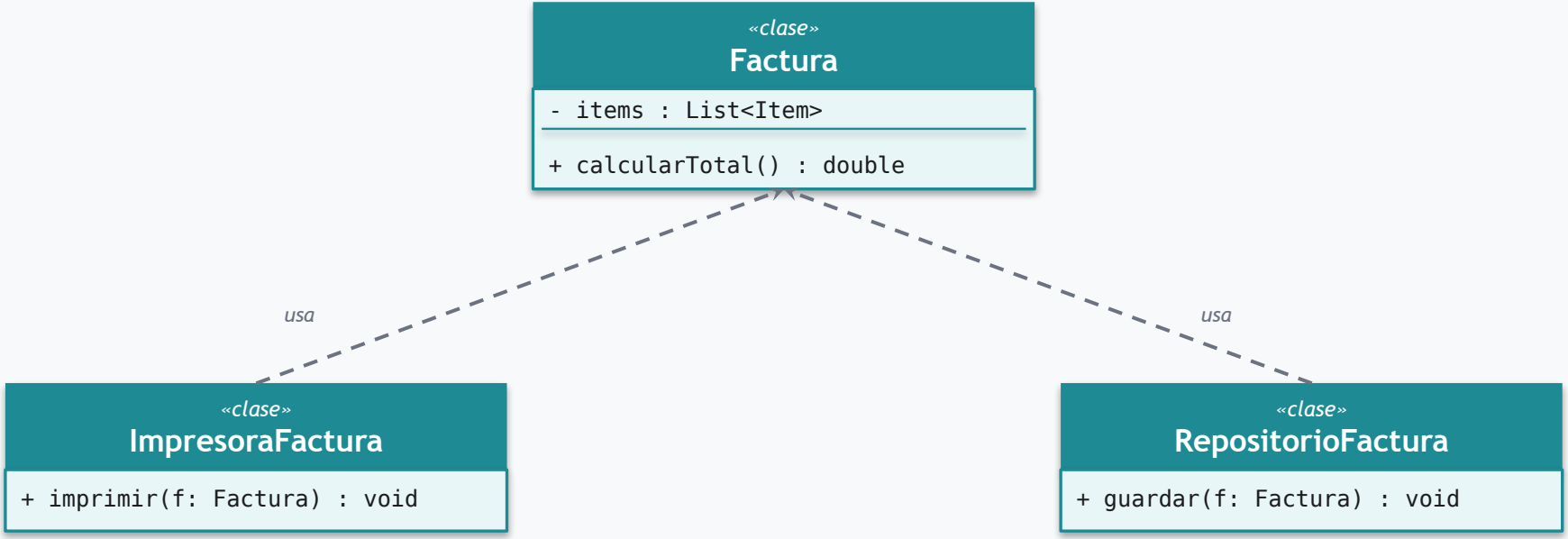
SRP — DIAGRAMA: ANTES (VIOLA EL PRINCIPIO)



PROBLEMA

La clase Factura tiene tres razones distintas para cambiar: si cambian las reglas de cálculo del total, si cambia el formato de impresión, o si cambia la forma de persistir los datos. Al mezclar esas responsabilidades en una sola clase, se viola el SRP.

SRP — DIAGRAMA: DESPUES (APLICA EL PRINCIPIO)



Cada clase tiene ahora una única responsabilidad: Factura solo calcula, ImpresoraFactura solo imprime y RepositorioFactura solo persiste.

SRP — CODIGO JAVA: ANTES vs DESPUES

ANTES (viola el principio)

```java
public class Factura {
    private List<Item> items;

    public double calcularTotal() {
        double total = 0;
        for (Item i : items) total +=
i.getPrecio();
        return total;
    }

    public void imprimir() {
        System.out.println("Total: " +
calcularTotal());
    }

    public void guardarEnBaseDeDatos() {
        // lógica de conexión y guardado
    }
}
```

DESPUES (aplica el principio)

```java
public class Factura {
    private List<Item> items;
    public double calcularTotal() { ... }
}

public class ImpresoraFactura {
    public void imprimir(Factura f) {
        System.out.println("Total: "
            + f.calcularTotal());
    }
}

public class RepositorioFactura {
    public void guardar(Factura f) {
        // lógica de conexión y guardado
    }
}
```

SRP — SEÑALES DE ALERTA (CODE SMELLS)

Estas señales en el código suelen indicar que se está violando el principio:

- ! El nombre de la clase es genérico o vago: Manager, Helper, Processor, Util.
- ! Cuesta describir qué hace la clase sin usar la palabra "y" (calcula Y imprime Y guarda).
- ! La clase importa librerías de dominios muy distintos (persistencia, UI, red) al mismo tiempo.
- ! Cambios pedidos por áreas distintas (ventas, IT, contable) terminan tocando la misma clase.
- ! Los tests de la clase fallan por razones que no tienen relación entre sí.

Tip: si necesitás un "y" para describir la clase, probablemente tiene más de una responsabilidad.

SRP — BENEFICIOS Y RIESGOS

AL APLICAR SRP

- ✓ Testing más simple y focalizado en cada responsabilidad.
- ✓ Cambios acotados y predecibles: se sabe qué clase tocar.
- ✓ Menor acoplamiento entre módulos del sistema.
- ✓ Clases más fáciles de nombrar, leer y entender.

AL IGNORAR SRP

- ✗ Aparecen clases "God Object" que crecen sin límite.
- ✗ Un cambio rompe funcionalidad que no tiene relación aparente.
- ✗ Es difícil escribir tests unitarios acotados.
- ✗ Los nuevos desarrolladores tardan más en entender la clase.

OCP — DEFINICIÓN Y RELACIÓN CON LA POO

Principio Abierto / Cerrado (Open / Closed Principle)

El software debe estar abierto a extensión pero cerrado a modificación: se agrega comportamiento nuevo sin alterar el código existente.

RELACIÓN CON LOS 4 PILARES DE LA POO

E

Encapsulamiento *(RELACIÓN LEVE)*

El detalle de cada variante nueva queda oculto detrás del contrato común.

A

Abstracción *(RELACIÓN FUERTE)*

Se define un contrato estable (interfaz o clase abstracta) que no cambia.

H

Herencia *(RELACIÓN MODERADA)*

Permite extender comportamiento creando subtipos sin tocar el código existente.

P

Polimorfismo *(RELACIÓN FUERTE)*

Cada variante nueva se agrega como una clase que implementa el contrato.

Ejemplo: agregar un tipo de cliente nuevo no debería obligar a modificar la clase que calcula descuentos.

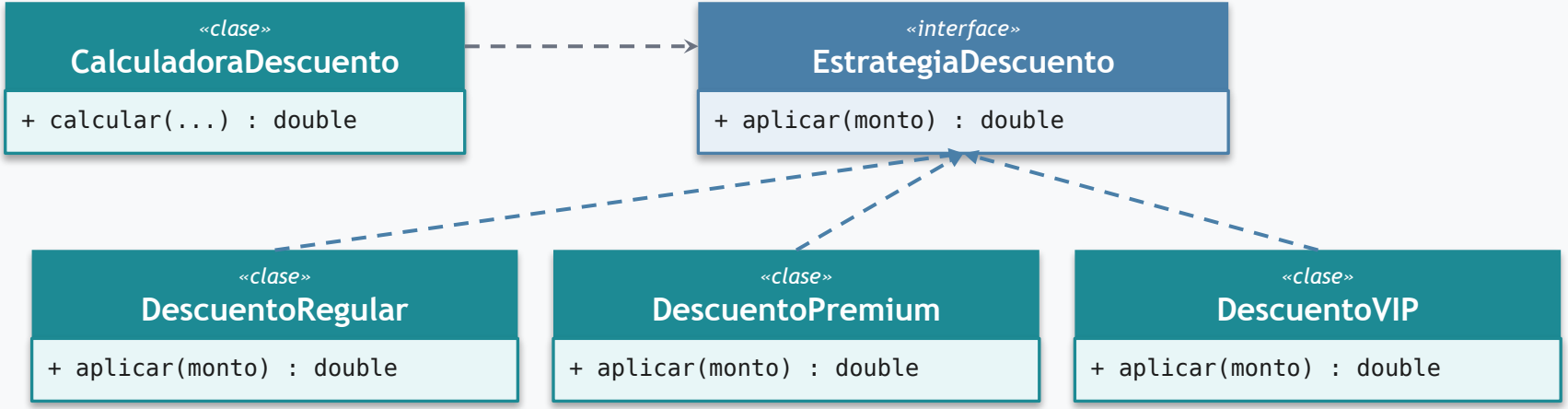
OCP — DIAGRAMA: ANTES (VIOLA EL PRINCIPIO)



PROBLEMA

Toda la lógica de descuentos vive en un único método con if/else por tipo de cliente. Agregar un nuevo tipo (por ejemplo VIP) obliga a modificar esta clase: no está cerrada a la modificación.

OCP — DIAGRAMA: DESPUES (APLICA EL PRINCIPIO)



CalculadoraDescuento y las estrategias concretas dependen de la abstracción EstrategiaDescuento. Para sumar DescuentoVIP alcanza con crear una clase nueva: no hace falta tocar el código existente.

OCP — CODIGO JAVA: ANTES vs DESPUES

ANTES (viola el principio)

```java
public class CalculadoraDescuento {
    public double calcular(
            String tipo, double monto) {
        if (tipo.equals("REGULAR")) {
            return monto * 0.95;
        } else if (tipo.equals("PREMIUM")) {
            return monto * 0.90;
        } else if (tipo.equals("VIP")) {
            return monto * 0.80;
        }
        return monto;
    }
}
```

DESPUES (aplica el principio)

```java
public interface EstrategiaDescuento {
    double aplicar(double monto);
}

public class DescuentoRegular
        implements EstrategiaDescuento {
    public double aplicar(double monto) {
        return monto * 0.95;
    }
}

public class CalculadoraDescuento {
    public double calcular(
            EstrategiaDescuento e, double monto) {
        return e.aplicar(monto);
    }
}
```

OCP — SEÑALES DE ALERTA (CODE SMELLS)

Estas señales en el código suelen indicar que se está violando el principio:

- ! Bloques if/else o switch que crecen cada vez que aparece un caso nuevo.
- ! Comentarios del tipo "// agregar acá el nuevo tipo" dentro del código.
- ! Cada funcionalidad nueva obliga a volver a testear código que ya funcionaba.
- ! Uso de instanceof o getClass() para decidir comportamiento según el tipo concreto.
- ! El historial de cambios muestra que la misma clase se modifica en cada sprint.

Tip: si para agregar un caso nuevo tenés que editar una clase existente, probablemente viola el OCP.

OCP — BENEFICIOS Y RIESGOS

AL APLICAR OCP

- ✓ Se agrega funcionalidad nueva sin re-testear lo existente.
- ✓ Menor riesgo de regresiones al extender el sistema.
- ✓ Las variantes nuevas quedan aisladas en clases propias.
- ✓ El código existente permanece estable en el tiempo.

AL IGNORAR OCP

- ✗ Cada cambio es una potencial regresión sobre lo que ya andaba.
- ✗ El código se vuelve frágil y "nadie lo quiere tocar".
- ✗ Los condicionales crecen sin control con cada caso nuevo.
- ✗ Cuesta razonar todas las combinaciones posibles del if/else.

LSP — DEFINICIÓN Y RELACIÓN CON LA POO

Principio de Sustitución de Liskov (Liskov Substitution Principle)

Los objetos de una subclase deben poder sustituir a los de su clase base sin alterar el comportamiento correcto del programa.

RELACIÓN CON LOS 4 PILARES DE LA POO

E — **Encapsulamiento** *(RELACIÓN LEVE)*
Las subclases no deben alterar invariantes internos que el resto del código da por sentado.

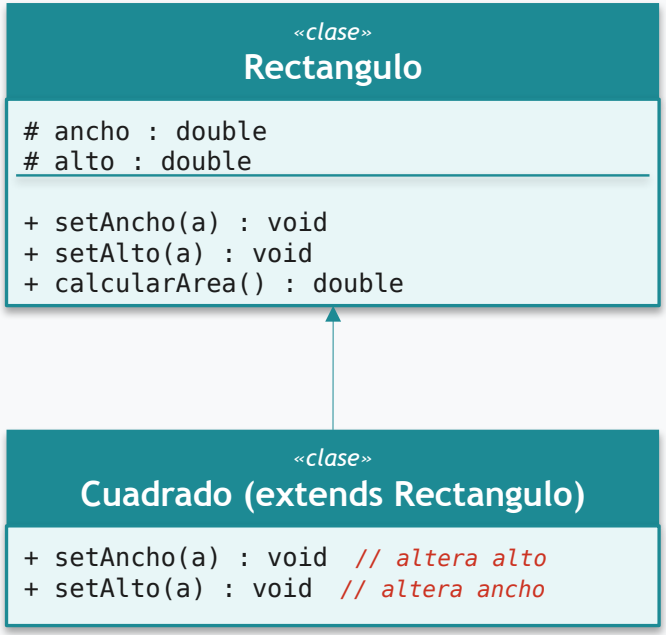
A — **Abstracción** *(RELACIÓN MODERADA)*
El contrato (pre y postcondiciones) del tipo base debe respetarse en toda la jerarquía.

H — **Herencia** *(RELACIÓN FUERTE)*
Define que significa 'ser un subtipo' correctamente: cumplir el contrato del tipo padre.

P — **Polimorfismo** *(RELACIÓN FUERTE)*
Garantiza que sustituir una subclase por otra no rompa el comportamiento del programa.

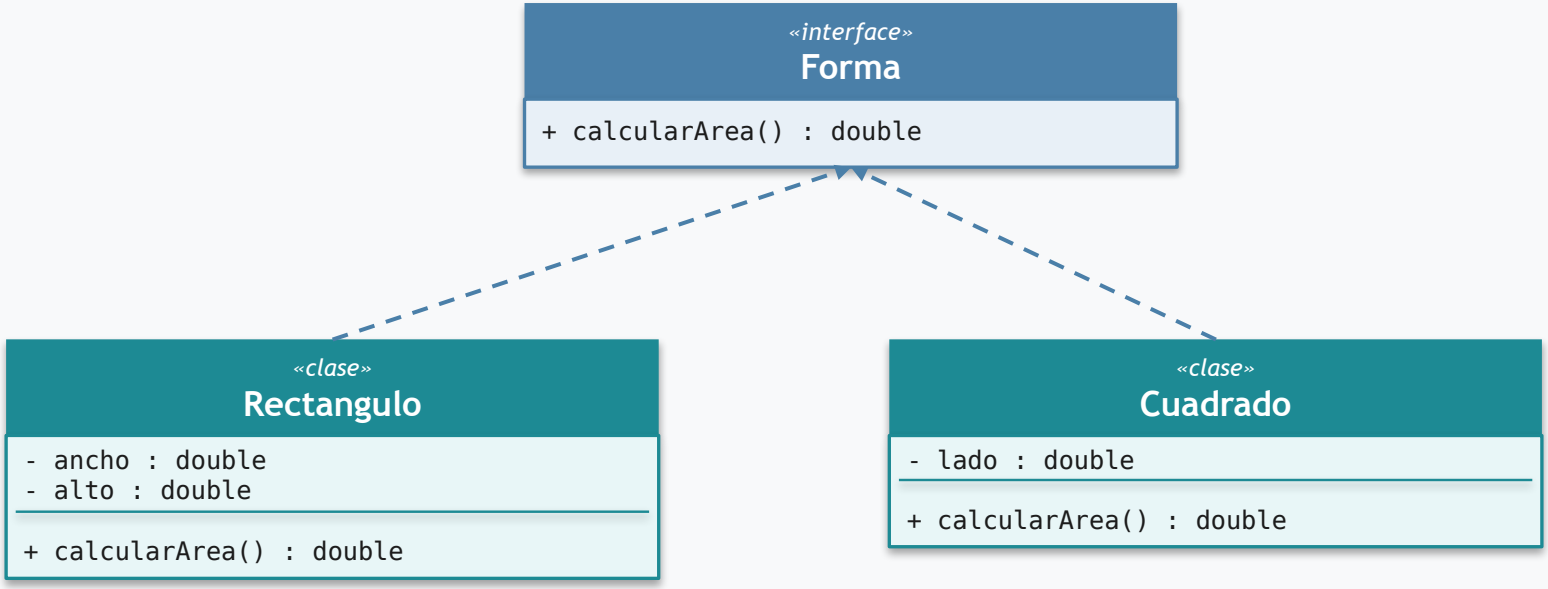
Ejemplo clásico: un Cuadrado que hereda de Rectangulo puede romper el comportamiento esperado.

LSP — DIAGRAMA: ANTES (VIOLA EL PRINCIPIO)



PROBLEMA: un código que espera un Rectangulo (ancho=5, alto=10, area=50) falla si recibe un Cuadrado, porque fijar el ancho también altera el alto. Cuadrado no sustituye a Rectangulo de forma segura: viola el LSP.

LSP — DIAGRAMA: DESPUES (APLICA EL PRINCIPIO)



Rectangulo y Cuadrado implementan Forma de manera independiente, sin heredar una de la otra. Cada clase garantiza sus propios invariantes: cualquier Forma puede sustituirse con seguridad.

LSP — CODIGO JAVA: ANTES vs DESPUES

ANTES (viola el principio)

```java
public class Rectangulo {
    protected double ancho;
    protected double alto;

    public void setAncho(double a) {
        this.ancho = a;
    }
    public void setAlto(double a) {
        this.alto = a;
    }
    public double calcularArea() {
        return ancho * alto;
    }
}

public class Cuadrado extends Rectangulo {
    public void setAncho(double a) {
        ancho = a; alto = a; // rompe LSP
    }
}
```

DESPUES (aplica el principio)

```java
public interface Forma {
    double calcularArea();
}

public class Rectangulo implements Forma {
    private double ancho;
    private double alto;
    public Rectangulo(double a, double h) {
        ancho = a; alto = h;
    }
    public double calcularArea() {
        return ancho * alto;
    }
}

public class Cuadrado implements Forma {
    private double lado;
    public Cuadrado(double lado) {
        this.lado = lado;
    }
    public double calcularArea() {
        return lado * lado;
    }
}
```

LSP — SEÑALES DE ALERTA (CODE SMELLS)

Estas señales en el código suelen indicar que se está violando el principio:

- ! Métodos sobreescritos que lanzan UnsupportedOperationException o no hacen nada.
- ! Uso de instanceof para chequear la subclase real antes de operar sobre ella.
- ! Precondiciones más estrictas o postcondiciones más débiles en la subclase.
- ! Comentarios del tipo "cuidado, esta subclase no funciona igual que las demás".
- ! Tests que fallan solo cuando se usa una subclase específica en vez de la base.

Tip: si necesitás preguntar de qué subclase se trata para usarla bien, algo viola el LSP.

LSP — BENEFICIOS Y RIESGOS

AL APLICAR LSP

- ✓ Polimorfismo confiable: cualquier subclase funciona donde se espera la base.
- ✓ El código cliente no necesita conocer las subclases concretas.
- ✓ Jerarquías de herencia más simples de razonar y mantener.
- ✓ Menos sorpresas al extender el sistema con nuevos subtipos.

AL IGNORAR LSP

- ✗ Aparecen bugs sutiles que solo ocurren con ciertas subclases.
- ✗ Hay que agregar chequeos de tipo para "parchear" el comportamiento.
- ✗ La jerarquía de herencia se vuelve frágil ante cualquier cambio.
- ✗ El polimorfismo deja de ser confiable para quien usa la clase base.

ISP — DEFINICIÓN Y RELACIÓN CON LA POO

Principio de Segregación de Interfaces (Interface Segregation Principle)

Ningún cliente debería verse obligado a depender de métodos que no usa: conviene tener varias interfaces específicas antes que una sola interfaz general.

RELACIÓN CON LOS 4 PILARES DE LA POO

E **Encapsulamiento** *(RELACIÓN FUERTE)*
Cada cliente conoce y depende solo de los métodos que realmente necesita.

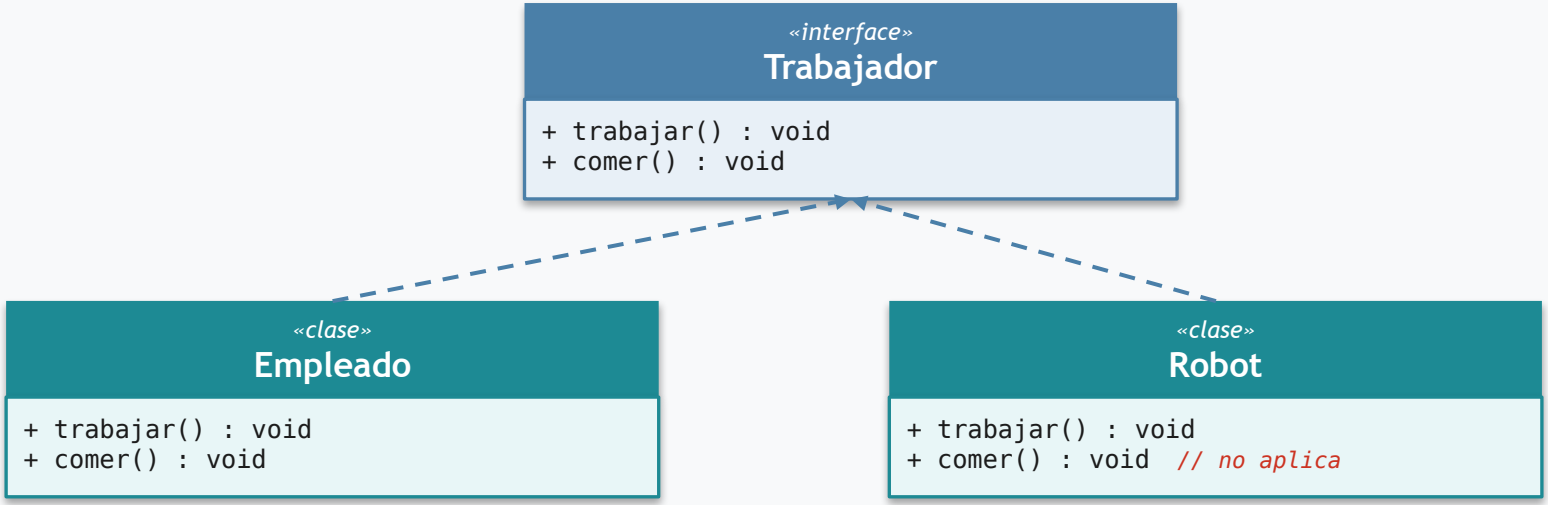
A **Abstracción** *(RELACIÓN FUERTE)*
Divide un contrato grande en contratos pequeños y cohesivos por necesidad.

H **Herencia** *(RELACIÓN LEVE)*
Evita forzar a una jerarquía entera a heredar métodos que no le corresponden.

P **Polimorfismo** *(RELACIÓN MODERADA)*
Interfaces chicas facilitan implementaciones más simples y sustituibles.

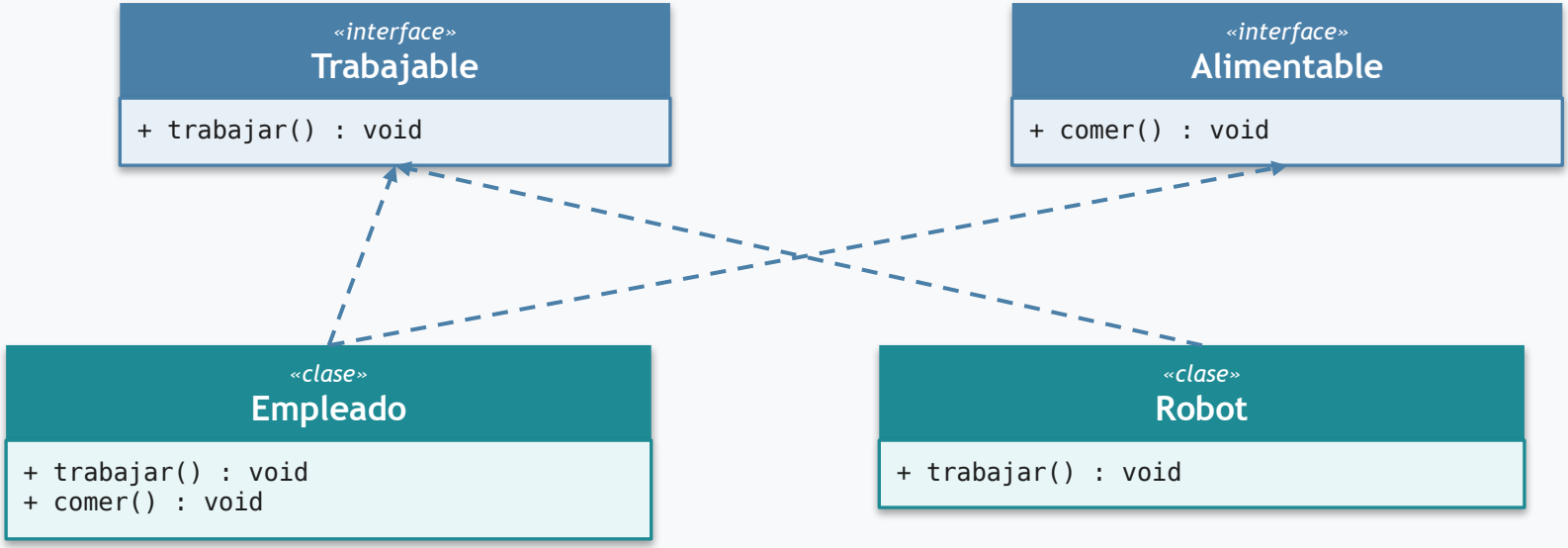
Ejemplo: una interfaz Trabajador con trabajar() y comer() obliga a un Robot a implementar algo que no le corresponde.

ISP — DIAGRAMA: ANTES (VIOLA EL PRINCIPIO)



PROBLEMA: Robot se ve forzado a implementar comer(), un método que no tiene sentido para el. La interfaz Trabajador es demasiado grande (interfaz 'gorda') y viola el ISP.

ISP — DIAGRAMA: DESPUES (APLICA EL PRINCIPIO)



Empleado implementa ambas interfaces; Robot solo implementa Trabajable. Cada clase depende únicamente de lo que necesita: ya no hay métodos forzados sin sentido.

ISP — CODIGO JAVA: ANTES vs DESPUES

ANTES (viola el principio)

```java
public interface Trabajador {
    void trabajar();
    void comer();
}

public class Empleado
        implements Trabajador {
    public void trabajar() { ... }
    public void comer() { ... }
}

public class Robot implements Trabajador {
    public void trabajar() { ... }
    public void comer() {
        throw new
            UnsupportedOperationException(
                "Un robot no come");
    }
}
```

DESPUES (aplica el principio)

```java
public interface Trabajable {
    void trabajar();
}

public interface Alimentable {
    void comer();
}

public class Empleado implements
        Trabajable, Alimentable {
    public void trabajar() { ... }
    public void comer() { ... }
}

public class Robot
        implements Trabajable {
    public void trabajar() { ... }
}
```

ISP — SEÑALES DE ALERTA (CODE SMELLS)

Estas señales en el código suelen indicar que se está violando el principio:

- ! Clases que implementan métodos de la interfaz dejándolos vacíos o con excepciones.
- ! Interfaces con muchos métodos que no todos los clientes llegan a usar.
- ! Comentarios del tipo "este método no aplica para esta clase".
- ! Cambiar la interfaz obliga a tocar clases que no usan el método modificado.
- ! Clientes que dependen de métodos que nunca invocan realmente.

Tip: si una clase implementa un método solo para no romper la interfaz, esa interfaz es demasiado grande.

ISP — BENEFICIOS Y RIESGOS

AL APLICAR ISP

- ✓ Clases más simples, con menos dependencias innecesarias.
- ✓ Cambios más aislados: modificar una interfaz chica afecta a menos clases.
- ✓ Contratos más claros y fáciles de implementar correctamente.
- ✓ Clientes que dependen solo de lo que realmente necesitan.

AL IGNORAR ISP

- ✗ Implementaciones vacías o que lanzan excepciones sin sentido.
- ✗ Clientes acoplados a métodos que jamás utilizan.
- ✗ Interfaces gigantes, difíciles de mantener y de entender.
- ✗ Cualquier cambio en la interfaz impacta a clases no relacionadas.

DIP — DEFINICIÓN Y RELACIÓN CON LA POO

Principio de Inversión de Dependencias (Dependency Inversion Principle)

Los módulos de alto nivel no deben depender de módulos de bajo nivel: ambos deben depender de abstracciones.

RELACIÓN CON LOS 4 PILARES DE LA POO

E **Encapsulamiento** *(RELACIÓN MODERADA)*
Los detalles de la implementación concreta quedan ocultos detrás de la abstracción.

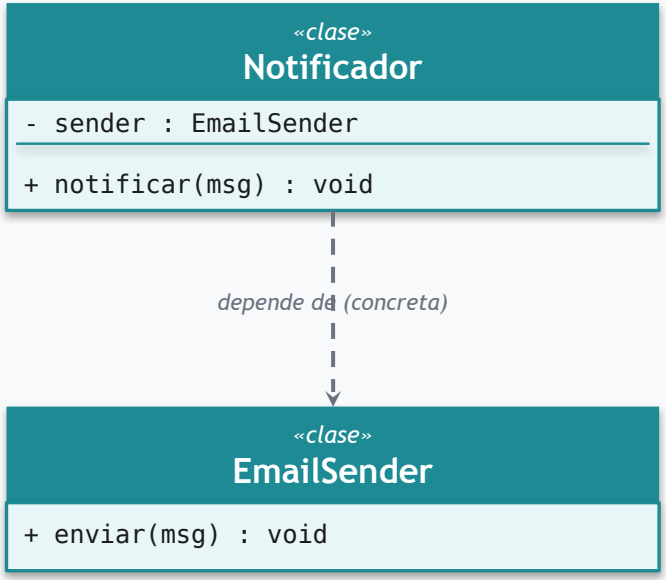
A **Abstracción** *(RELACIÓN FUERTE)*
Los módulos de alto nivel dependen de una interfaz, no de una clase concreta.

H **Herencia** *(RELACIÓN LEVE)*
La abstracción suele modelarse como interfaz o clase de la que heredan las implementaciones.

P **Polimorfismo** *(RELACIÓN FUERTE)*
Cualquier implementación de esa interfaz puede inyectarse en tiempo de ejecución.

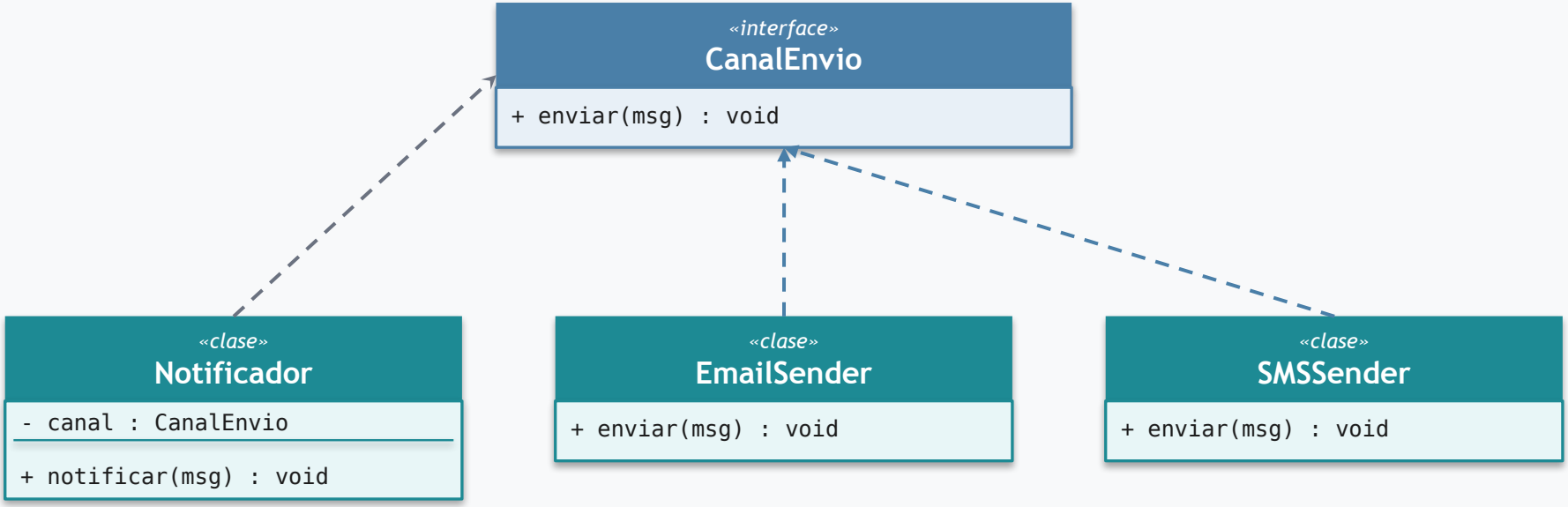
Ejemplo: un Notificador que crea directamente un EmailSender queda atado a ese canal concreto.

DIP — DIAGRAMA: ANTES (VIOLA EL PRINCIPIO)



PROBLEMA: Notificador (módulo de alto nivel) depende directamente de EmailSender (detalle concreto). No se puede cambiar de canal ni testear sin modificar Notificador.

DIP — DIAGRAMA: DESPUES (APLICA EL PRINCIPIO)



Notificador y los canales concretos dependen de la abstracción CanalEnvio. Agregar un canal nuevo (por ejemplo push) no exige modificar Notificador, y en los tests es fácil reemplazar la implementación real por una simulada.

DIP — CODIGO JAVA: ANTES vs DESPUES

ANTES (viola el principio)

```java
public class EmailSender {
    public void enviar(String msg) {
        System.out.println("Email: " + msg);
    }
}

public class Notificador {
    private EmailSender sender =
            new EmailSender();

    public void notificar(String msg) {
        sender.enviar(msg);
    }
}
```

DESPUES (aplica el principio)

```java
public interface CanalEnvio {
    void enviar(String msg);
}

public class EmailSender
        implements CanalEnvio {
    public void enviar(String msg) {
        System.out.println("Email: " + msg);
    }
}

public class Notificador {
    private final CanalEnvio canal;

    public Notificador(CanalEnvio canal) {
        this.canal = canal;
    }
    public void notificar(String msg) {
        canal.enviar(msg);
    }
}
```

DIP — SEÑALES DE ALERTA (CODE SMELLS)

Estas señales en el código suelen indicar que se está violando el principio:

- ! Uso de "new ClaseConcreta()" dentro de la lógica de negocio, en vez de recibir la dependencia.
- ! Dificultad para testear con mocks o stubs porque las dependencias están hardcodeadas.
- ! Cambiar de proveedor (por ejemplo de base de datos) obliga a modificar muchas clases.
- ! Módulos de alto nivel que importan detalles concretos de bajo nivel (drivers, SDKs).
- ! Constructores que crean sus propias dependencias en vez de recibirlas.

Tip: si una clase hace "new" de sus propias dependencias, probablemente conviene inyectarlas.

DIP — BENEFICIOS Y RIESGOS

AL APLICAR DIP

- ✓ Fácil de testear: se puede inyectar un mock o stub en los tests.
- ✓ Fácil de cambiar la implementación concreta sin tocar el módulo de alto nivel.
- ✓ Menor acoplamiento entre las capas del sistema.
- ✓ Las dependencias quedan explícitas en el constructor o los parámetros.

AL IGNORAR DIP

- ✗ Tests lentos o imposibles de aislar sin infraestructura real.
- ✗ Cambiar de proveedor externo resulta muy costoso.
- ✗ El módulo de alto nivel termina sabiendo demasiado de detalles técnicos.
- ✗ El acoplamiento directo dificulta reutilizar la lógica de negocio.



DISEÑAR CON SOLID ES DISEÑAR PARA EL CAMBIO

“Menos acoplamiento, mayor claridad, código que dura.”
Los cinco principios SOLID ayudan a escribir software más simple de mantener y de extender.